



Universidad Veracruzana

Autorización con grupos en Django

Usuarios, sesiones, roles y decoradores

Programación segura

Dr. Juan Manuel Gutiérrez Méndez

Propósito

Comprender cómo se implementa la autorización en Django usando usuarios, sesiones, grupos, roles, relaciones entre modelos y decoradores.

La autorización define qué puede hacer un usuario autenticado y qué datos puede consultar.

Contexto del proyecto

En la aplicación logística existen dos tipos principales de usuario:

Rol	Necesidad
Cliente	Registrar y consultar sus propios bienes
Supervisor	Consultar todos los bienes registrados
Usuario sin sesión	No debe acceder a vistas protegidas

La autorización debe proteger tanto las vistas como los datos.

Problema de seguridad

Un cliente podría intentar:

- abrir una URL protegida;
- modificar el identificador de un bien;
- consultar bienes de otro cliente;
- acceder a reportes del supervisor.

Ocultar opciones en la interfaz no impide que un usuario intente acceder directamente por URL.

Autenticación, sesión y autorización

Concepto	Pregunta que responde	Ejemplo
Autenticación	¿Quién eres?	Inicio de sesión
Sesión	¿Cómo se mantiene tu identidad?	Cookie de sesión
Autorización	¿Qué puedes hacer?	Ver solo bienes propios
Rol	¿Qué responsabilidad tienes?	Cliente o supervisor
Alcance de datos	¿Qué registros te pertenecen?	Bienes asociados al cliente

Autenticación

La autenticación confirma la identidad del usuario.

En Django se realiza mediante:

- usuario;
- contraseña;
- formulario de inicio de sesión;
- sesión creada después del login.

Una vez autenticado, la vista puede acceder al usuario actual con:

```
request.user
```

Sesión

La sesión permite mantener la identidad del usuario durante la navegación.

```
Usuario inicia sesión
  |
  v
Django crea una sesión
  |
  v
El navegador guarda una cookie
  |
  v
Cada solicitud permite identificar al usuario
```

La sesión permite que Django recuerde al usuario autenticado entre solicitudes.

¿Dónde se guardan las sesiones?

En una configuración común, Django guarda las sesiones en la base de datos.

La tabla utilizada es:

```
django_session
```

El navegador no guarda todos los datos de sesión.

El navegador guarda una cookie con el identificador de sesión.

Cookie de sesión

La cookie funciona como una referencia.

Navegador

guarda: `sessionid`

Servidor Django

busca la sesión en `django_session`

Así Django puede relacionar cada solicitud con el usuario autenticado.

Autorización

La autorización ocurre después de la autenticación.

El usuario ya inició sesión.
Ahora el sistema debe decidir:

- ¿Puede registrar bienes?
- ¿Puede ver este bien?
- ¿Puede consultar todos los bienes?

Autenticarse no significa tener acceso a toda la información.

Usuarios, grupos y roles

Django incluye usuarios y grupos.

Elemento	Función
User	Representa al usuario que inicia sesión
Group	Agrupar usuarios con una responsabilidad común
Rol	Interpretación funcional del grupo
Permiso	Acción específica permitida

En esta práctica, los grupos representarán roles.

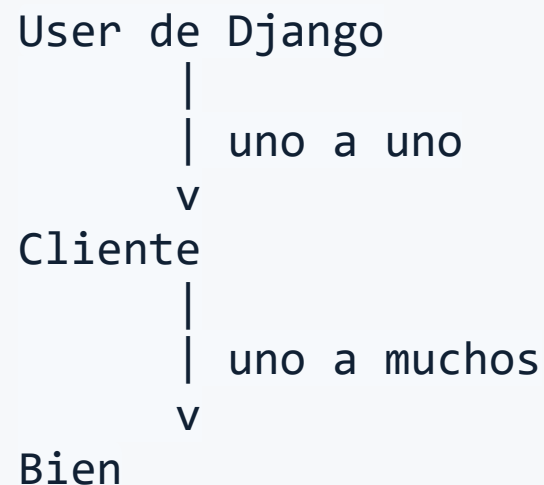
Roles del proyecto

Grupo de Django	Rol funcional	Alcance
cliente	Cliente de la empresa logística	Sus propios bienes
supervisor	Supervisor operativo	Todos los bienes
Sin grupo válido	Usuario no autorizado	Sin acceso funcional

Relación de datos necesaria

Para que la autorización funcione, no basta con saber el rol del usuario.

También se debe saber qué datos le pertenecen.



El grupo define el rol del usuario; la relación entre modelos define el alcance de los datos.

User, Cliente y Bien

Relación	Propósito
User → Cliente	Saber qué cliente inició sesión
Cliente → Bien	Saber a quién pertenece cada bien
request.user → cliente	Obtener el cliente del usuario actual
cliente → bienes	Consultar solo los bienes propios

Esta estructura permite aplicar autorización basada en propietario.

Ejemplo conceptual

Cuando un cliente inicia sesión:

```
request.user  
  |  
  v  
Cliente asociado  
  |  
  v  
Bienes del cliente
```

La consulta segura se construye usando esa relación:

```
Bien.objects.filter(cliente__usuario=request.user)
```

Supervisor y alcance global

El supervisor no consulta bienes por propietario.

Su rol permite una vista general:

```
Bien.objects.all()
```

La diferencia está en la regla de autorización:

Usuario	Consulta permitida
Cliente	Bienes asociados a su cliente
Supervisor	Todos los bienes
Sin rol válido	Ningún bien

Flujo general de autorización

1. Solicitud

El usuario solicita una vista.

2. Sesión

Django verifica si inició sesión.

3. Rol

Se revisa si es cliente o supervisor.

4. Relación

Se identifica qué datos le pertenecen.

5. Respuesta

Se entrega solo información permitida.

Funciones auxiliares para roles

Para no repetir lógica en todas las vistas, conviene crear funciones auxiliares.

```
seguridad/roles.py
```

Ejemplos:

```
es_cliente(user)  
es_supervisor(user)
```

Las funciones auxiliares concentran reglas de rol y hacen más legible el código.

Ejemplo de roles.py

```
# seguridad/roles.py

GRUPO_CLIENTE = "cliente"
GRUPO_SUPERVISOR = "supervisor"

def pertenece_a_grupo(user, nombre_grupo):
    return user.is_authenticated and user.groups.filter(
        name=nombre_grupo
    ).exists()

def es_cliente(user):
    return pertenece_a_grupo(user, GRUPO_CLIENTE)

def es_supervisor(user):
    return pertenece_a_grupo(user, GRUPO_SUPERVISOR)
```

Por qué usar funciones auxiliares

Sin funciones auxiliares	Con funciones auxiliares
Se repite código en cada vista	La regla se concentra en un lugar
La intención queda dispersa	La intención se lee directamente
Es más fácil cometer errores	Es más fácil mantener el proyecto
Cambios futuros son costosos	Cambios futuros son más simples

Decoradores en Django

Un decorador permite ejecutar una regla antes de la vista.

Ejemplo:

```
@login_required
def lista_bienes(request):
    ...
```

Antes de ejecutar `lista_bienes`, Django revisa si el usuario tiene sesión activa.

@login_required

`@login_required` protege una vista para que solo pueda ser usada por usuarios autenticados.

```
from django.contrib.auth.decorators import login_required

@login_required
def reporte_bienes(request):
    ...
```

Si el usuario no ha iniciado sesión, Django lo redirige al login configurado.

Decoradores propios

Además de `@login_required`, se pueden crear decoradores propios.

```
@cliente_requerido  
def crear_bien(request):  
    ...
```

```
@supervisor_requerido  
def reporte_general(request):  
    ...
```

Un decorador permite convertir una regla de autorización en una protección reutilizable para varias vistas.

Cómo funciona un decorador

```
Vista original
|
v
Decorador recibe la vista
|
v
Wrapper revisa la regla de seguridad
├─ Si cumple: ejecuta la vista
└─ Si no cumple: bloquea el acceso
```

El decorador no reemplaza la vista; la envuelve con una verificación previa.

Decorador para cliente

```
# seguridad/decorators.py

from functools import wraps
from django.core.exceptions import PermissionDenied
from .roles import es_cliente

def cliente_requerido(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not es_cliente(request.user):
            raise PermissionDenied("Se requiere rol de cliente.")

        return view_func(request, *args, **kwargs)

    return wrapper
```

Decorador para supervisor

```
# seguridad/decorators.py

from functools import wraps
from django.core.exceptions import PermissionDenied
from .roles import es_supervisor

def supervisor_requerido(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not es_supervisor(request.user):
            raise PermissionDenied("Se requiere rol de supervisor.")

        return view_func(request, *args, **kwargs)

    return wrapper
```

¿Qué es `@wraps(view_func)`?

`@wraps(view_func)` se usa dentro de un decorador para conservar información de la función original.

Conserva, por ejemplo:

- el nombre de la vista;
- su documentación;
- metadatos útiles para depuración;
- mejor compatibilidad con Django y otras herramientas.

`@wraps(view_func)` no realiza la autorización; ayuda a que la función decorada conserve su identidad original.

Uso combinado de decoradores

```
@login_required
@cliente_requerido
def crear_bien(request):
    ...
```

La lectura conceptual es:

1. El usuario debe tener sesión activa.
2. El usuario debe tener rol de cliente.
3. Si cumple, puede registrar un bien.

Supervisor protegido

```
@login_required
@supervisor_requerido
def reporte_general_bienes(request):
    ...
```

La lectura conceptual es:

1. El usuario debe tener sesión activa.
2. El usuario debe tener rol de supervisor.
3. Si cumple, puede consultar el reporte general.

Autorización en vistas

La vista debe decidir qué información puede consultar el usuario.

```
if es_supervisor(request.user):  
    bienes = Bien.objects.all()  
  
elif es_cliente(request.user):  
    bienes = Bien.objects.filter(  
        cliente__usuario=request.user  
    )
```

La consulta al ORM debe aplicar el alcance permitido.

Filtrado por propietario

El cliente solo debe ver bienes asociados a su usuario.

```
Bien.objects.filter(cliente__usuario=request.user)
```

El supervisor puede consultar todos los bienes.

```
Bien.objects.all()
```

La autorización segura se aplica en la consulta, no después de mostrar los datos.

Acceso directo por URL

Un cliente podría intentar abrir una ruta como:

```
/bienes/25/
```

Si el bien pertenece a otro cliente, el sistema debe bloquear el acceso.

Ejemplo de consulta segura:

```
get_object_or_404(  
    Bien,  
    id=bien_id,  
    cliente__usuario=request.user  
)
```


Interfaz vs backend

INTERFAZ

- Puede ocultar opciones.
- Mejora la experiencia de usuario.
- No detiene acceso directo por URL.
- No debe ser el único control.

BACKEND

- Valida sesión.
- Valida rol.
- Filtra datos.
- Debe impedir accesos no autorizados.

Autorización en templates

Los templates pueden mostrar opciones según el rol.

```
{% if es_supervisor %}  
  <a href="{% url 'reporte_general_bienes' %}">  
    Ver reporte general  
  </a>  
{% endif %}
```

Esto mejora la navegación, pero no sustituye la validación en la vista.

Configuración básica de sesión

Django permite configurar rutas relacionadas con el inicio y cierre de sesión.

```
LOGIN_URL = "login"  
LOGIN_REDIRECT_URL = "lista_bienes"  
LOGOUT_REDIRECT_URL = "login"
```

También se pueden proteger cookies:

```
SESSION_COOKIE_HTTPONLY = True  
SESSION_COOKIE_SECURE = True
```

En producción, `SESSION_COOKIE_SECURE` debe usarse con HTTPS.

Roles en la app logística

Vista o acción	Cliente	Supervisor
Iniciar sesión	Sí	Sí
Registrar bien	Sí, propio	No requerido en esta práctica
Ver reporte propio	Sí	Sí, si se habilita
Ver todos los bienes	No	Sí
Ver bien ajeno	No	Sí
Acceder sin sesión	No	No

Regla conceptual completa

Para autorizar correctamente una vista, se deben validar tres niveles:

Nivel	Pregunta
Sesión	¿El usuario inició sesión?
Rol	¿Es cliente o supervisor?
Alcance de datos	¿El recurso le pertenece o tiene permiso global?

Sesión, rol y propiedad del dato deben trabajar juntos.

Relación con el módulo seguridad

En el proyecto, `seguridad/` concentra reglas transversales.

```
seguridad/  
├── roles.py  
├── decorators.py  
├── tokens.py  
└── validaciones.py
```

Para autorización con usuarios se usarán principalmente:

- `roles.py` ;
- `decorators.py` .

Errores comunes

Error	Riesgo
Usar solo botones ocultos	Acceso directo por URL
No relacionar <code>Cliente</code> con <code>User</code>	No se identifica al propietario
No relacionar <code>Bien</code> con <code>Cliente</code>	No se puede filtrar por usuario
No usar decoradores	Reglas repetidas o incompletas
No filtrar por <code>request.user</code>	Exposición de bienes ajenos
Usar <code>is_staff</code> como rol funcional	Mezcla administración con operación

Pruebas conceptuales necesarias

Caso	Resultado esperado
Usuario sin sesión abre /bienes/	Redirección a login
Cliente A consulta lista	Solo ve bienes de Cliente A
Cliente A intenta ver bien de Cliente B	Acceso denegado o 404
Supervisor consulta lista	Ve todos los bienes
Cliente intenta abrir reporte general	Acceso denegado
Template oculta opciones no permitidas	Interfaz coherente con el rol

Síntesis

La autorización segura combina sesión activa, grupo del usuario, decoradores reutilizables y relaciones de datos que permiten filtrar por propietario.

En la aplicación logística, el grupo indica si el usuario es cliente o supervisor; la relación `User → Cliente → Bien` define qué información puede consultar.

Transición

La siguiente práctica implementará esta lógica paso a paso:

- app `clientes` ;
- relación `Cliente → User` ;
- relación `Bien → Cliente` ;
- grupos `cliente` y `supervisor` ;
- funciones en `seguridad/roles.py` ;
- decoradores en `seguridad/decorators.py` ;
- vistas y reportes protegidos.

Gracias
¿Preguntas?

